

BÁO CÁO

Họ tên: Cao Hữu Phúc

MSSV: 20235398

Bài 1:

Bài tập 03.01

Sử dụng kỹ thuật đa tiến trình, lập trình ứng dụng **file server** đơn giản như sau:

- Khi client mới được chấp nhận, server sẽ gửi danh sách các file cho client. Các file này trong thư mục được thiết lập trên server. Dòng đầu danh sách là chuỗi “**OK N\r\n**” nếu có N file trong thư mục. Các tên file phân cách nhau bởi ký tự **\r\n**, kết thúc danh sách bởi ký tự **\r\n\r\n**.
- Nếu không có file nào trong thư mục thì server gửi chuỗi “**ERROR No files to download\r\n**” rồi đóng kết nối.
- Client gửi tên file để tải về, server kiểm tra nếu file tồn tại thì gửi chuỗi “**OK N\r\n**” trong đó N là kích thước file, sau đó là nội dung file cho client và đóng kết nối, nếu file không tồn tại thì gửi thông báo lỗi và yêu cầu client gửi lại tên file.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <sys/socket.h>
```

```
#include <arpa/inet.h>
```

```
#include <sys/wait.h>
```

```
#include <signal.h>
```

```
#include <dirent.h>
```

```
#include <sys/stat.h>
```

```
#define SHARED_DIR "./shared_folder"
```

```
void signalHandler(int signo) {
```

```
    wait(NULL);
```

```
}
```

```
int main() {
```

```
    int listener = socket(AF_INET, SOCK_STREAM,  
IPPROTO_TCP);
```

```
    struct sockaddr_in addr;
```

```
    addr.sin_family = AF_INET;
```

```
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
```

```
    addr.sin_port = htons(9000);
```

```
    bind(listener, (struct sockaddr *)&addr, sizeof(addr));
```

```
    listen(listener, 5);
```

```
signal(SIGCHLD, signalHandler);
printf("File Server dang chay o cong 9000...\n");
while(1){
    int client = accept(listener, NULL, NULL);
    if (client == -1) continue;
    if (fork() == 0) {
        close(listener);
        DIR *d = opendir(SHARED_DIR);
        struct dirent *dir;
        char file_list[2048] = "";
        int file_count = 0;
        if(d){
            while ((dir = readdir(d)) != NULL) {
                if (dir->d_type == DT_REG) {
                    strcat(file_list, dir->d_name);
                    strcat(file_list, "\r\n");
                    file_count++;
                }
            }
        }
        closedir(d);
    }
}
```

```

}
if (file_count == 0) {
    char *err = "ERROR No files to download\r\n";
    send(client, err, strlen(err), 0);
    close(client);
    exit(0);
} else {
    char header[64];
    sprintf(header, "OK %d\r\n", file_count);
    send(client, header, strlen(header), 0);
    send(client, file_list, strlen(file_list), 0);
    send(client, "\r\n", 2, 0);
}
char buf[256];
while(1){
    int bytes = recv(client, buf, sizeof(buf) - 1, 0);
    if (bytes <= 0) break;
    buf[bytes] = '\0';
    buf[strcspn(buf, "\r\n")] = 0;
    char filepath[512];

```

```

sprintf(filepath, "%s/%s", SHARED_DIR, buf);
FILE *f = fopen(filepath, "rb");
if(f){
    struct stat st;
    stat(filepath, &st);
    char res_header[64];
    sprintf(res_header, "OK %ld\r\n", st.st_size);
    send(client, res_header, strlen(res_header), 0);
    char file_data[1024];
    int read_bytes;
    while((read_bytes = fread(file_data, 1,
sizeof(file_data), f)) > 0){
        send(client, file_data, read_bytes, 0);
    }
    fclose(f);
    break;
} else {
    char *err = "ERROR File not found\r\nVui long
nhap lai ten file: ";
    send(client, err, strlen(err), 0);
}

```

```
    }  
    close(client);  
    exit(0);  
}  
close(client);  
}  
return 0;  
}
```

```
huuphuc1@Phuc:~$ gcc -o file_server file_server.c  
❖ huuphuc1@Phuc:~$ ./file_server  
file Server dang chay o cong 9000...
```

Server

```
huyphuc10@huc:~$ nc 127.0.0.1 9000
OK 2
test1.txt
test2.txt
test3.txt
OK 21
Nội dung test file 1
[]

huyphuc10@huc:~$ nc 127.0.0.1 9000
OK 2
test1.txt
test2.txt
test3.txt
error file not found
Vui lòng nhập lại tên file: test1.txt
OK 21
Nội dung test file 1
[]
```

Client

Bài 2:

Bài tập 03.02

Sử dụng kỹ thuật đa luồng, lập trình ứng dụng chat nhóm 2 người như sau:

- Mỗi client khi kết nối đến server sẽ được lưu vào hàng đợi, nếu số lượng client trong hàng đợi là 2 thì 2 client này sẽ được ghép cặp với nhau.
- Khi 2 client đã được ghép cặp thì tin nhắn gửi từ client này sẽ được chuyển tiếp sang client kia và ngược lại.
- Nếu 1 trong 2 client ngắt kết nối thì client còn lại cũng được ngắt kết nối.

```
#include <stdio.h>
```

```
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <pthread.h>
#include <poll.h>
```

```
struct Pair {
    int client1;
    int client2;
};
```

```
void* pair_chat_handler(void* arg) {
    struct Pair *p = (struct Pair*)arg;
    struct pollfd fds[2];
    char buf[1024];
    fds[0].fd = p->client1; fds[0].events = POLLIN;
    fds[1].fd = p->client2; fds[1].events = POLLIN;
    char *msg1 = "Da ghep cap! Ban co the bat dau chat.\n";
```



```
send(p->client1, msg1, strlen(msg1), 0);
send(p->client2, msg1, strlen(msg1), 0);
while(1){
    int ret = poll(fds, 2, -1);
    if(ret < 0) break;
    if(fds[0].revents & (POLLIN | POLLERR)){
        int bytes = recv(p->client1, buf, sizeof(buf)-1, 0);
        if (bytes <= 0) break;
        buf[bytes] = '\0';
        send(p->client2, buf, bytes, 0);
    }
    if(fds[1].revents & (POLLIN | POLLERR)) {
        int bytes = recv(p->client2, buf, sizeof(buf)-1, 0);
        if (bytes <= 0) break;
        buf[bytes] = '\0';
        send(p->client1, buf, bytes, 0);
    }
}
printf(">> Mot cap chat da ngat ket noi.\n");
close(p->client1);
```

```

    close(p->client2);

    free(p);

    return NULL;
}

int main() {
    int listener = socket(AF_INET, SOCK_STREAM,
IPPROTO_TCP);

    struct sockaddr_in addr;

    addr.sin_family = AF_INET;

    addr.sin_addr.s_addr = htonl(INADDR_ANY);

    addr.sin_port = htons(9000);

    bind(listener, (struct sockaddr *)&addr, sizeof(addr));

    listen(listener, 10);

    printf("Pair Chat Server dang lang nghe o cong 9000...\n");

    while(1){
        printf("Dang doi Client 1...\n");

        int c1 = accept(listener, NULL, NULL);

        send(c1, "Dang cho nguoi ghep cap...\n", 27, 0);

        printf("Dang doi Client 2...\n");

        int c2 = accept(listener, NULL, NULL);
    }
}

```

```

    struct Pair *p = malloc(sizeof(struct Pair));

    p->client1 = c1;
    p->client2 = c2;

    pthread_t tid;

    pthread_create(&tid, NULL, pair_chat_handler, (void*)p);

    pthread_detach(tid);

}

return 0;

}

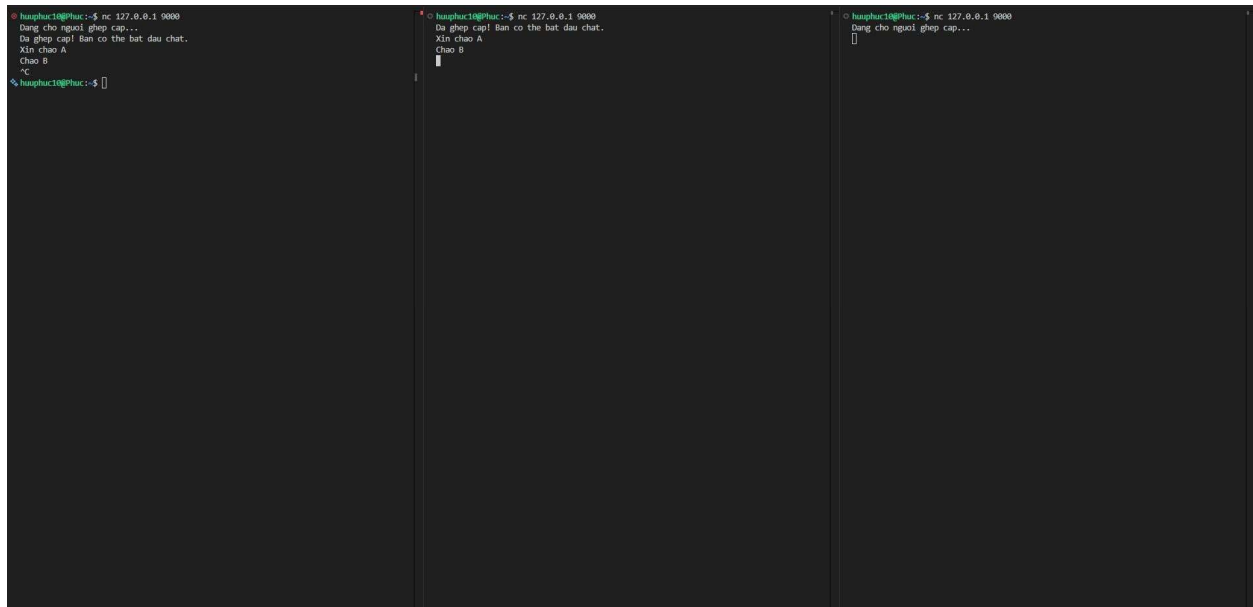
```

```

huuphuc1@Phuc:~$ gcc -pthread -o pair_chat pair_chat.c
huuphuc1@Phuc:~$ ./pair_chat
Pair Chat Server dang lang nghe o cong 9000...
Dang doi Client 1...
Dang doi Client 2...
Dang doi Client 1...
Dang doi Client 2...
>> Mot cap chat da ngat ket noi.

```

Server



Client

Bài 3:

Lập trình ứng dụng **chat_server** ([slide 173](#)) sử dụng kỹ thuật multithread.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <sys/socket.h>
```

```
#include <arpa/inet.h>
```

```
#include <pthread.h>
```

```
#define MAX_CLIENTS 64
```

```

struct ClientInfo {
    int fd;
    char id[32];
};

struct ClientInfo clients[MAX_CLIENTS];
int num_clients = 0;
pthread_mutex_t clients_mutex =
PTHREAD_MUTEX_INITIALIZER;

void broadcast_message(int sender_fd, char *sender_id, char
*msg) {
    char send_buf[1024];
    sprintf(send_buf, "%s: %s\n", sender_id, msg);
    pthread_mutex_lock(&clients_mutex);
    for (int i = 0; i < num_clients; i++) {
        if (clients[i].fd != sender_fd) {
            send(clients[i].fd, send_buf, strlen(send_buf), 0);
        }
    }
    pthread_mutex_unlock(&clients_mutex);
}

void remove_client(int fd) {

```

```

pthread_mutex_lock(&clients_mutex);
for (int i = 0; i < num_clients; i++) {
    if (clients[i].fd == fd) {
        for (int j = i; j < num_clients - 1; j++) {
            clients[j] = clients[j + 1];
        }
        num_clients--;
        break;
    }
}
pthread_mutex_unlock(&clients_mutex);
}

void* client_handler(void *arg) {
    int client_fd = *(int*)arg;
    free(arg);
    char buf[512];
    char client_id[32] = "";
    char client_name[64] = "";
    int logged_in = 0;

```

```

    char *prompt = "Vui long dang nhap theo cu phap 'client_id:
client_name'\n";

    send(client_fd, prompt, strlen(prompt), 0);

    while (!logged_in) {

        int bytes = recv(client_fd, buf, sizeof(buf) - 1, 0);

        if (bytes <= 0) {

            close(client_fd);

            return NULL;

        }

        buf[bytes] = '\0';

        buf[strcspn(buf, "\r\n")] = 0;

        int parsed = sscanf(buf, "%31[^:]: %63s", client_id,
client_name);

        if (parsed == 2) {

            logged_in = 1;

            char *success = ">> Dang nhap thanh cong! Ban co the
chat.\n";

            send(client_fd, success, strlen(success), 0);

            pthread_mutex_lock(&clients_mutex);

            clients[num_clients].fd = client_fd;

            strcpy(clients[num_clients].id, client_id);

```

```

        num_clients++;
        pthread_mutex_unlock(&clients_mutex);
        printf(">> Client [%s] da dang nhap.\n", client_id);
    } else {
        char *err = ">> Sai cu phap! Mau hop le: 'alice:
AliceNguyen'\n";
        send(client_fd, err, strlen(err), 0);
    }
}

while (1) {
    int bytes = recv(client_fd, buf, sizeof(buf) - 1, 0);
    if (bytes <= 0) {
        printf(">> Client [%s] da ngat ket noi.\n", client_id);
        remove_client(client_fd);
        break;
    }
    buf[bytes] = '\0';
    buf[strcspn(buf, "\r\n")] = 0;
    broadcast_message(client_fd, client_id, buf);
}

```



```

    close(client_fd);
    return NULL;
}

int main() {
    int listener = socket(AF_INET, SOCK_STREAM,
IPPROTO_TCP);
    struct sockaddr_in addr;
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons(9000);
    bind(listener, (struct sockaddr *)&addr, sizeof(addr));
    listen(listener, 10);
    printf("Chat Server Multithread dang lang nghe o cong
9000...\n");
    while (1) {
        int client_fd = accept(listener, NULL, NULL);
        if (client_fd == -1) continue;
        int *client_ptr = malloc(sizeof(int));
        *client_ptr = client_fd;
        pthread_t tid;
        pthread_create(&tid, NULL, client_handler, client_ptr);
    }
}

```

```

        pthread_detach(tid);
    }

    return 0;
}

```

```

huophuc1@Phuc:~$ gcc -pthread -o chat_server_multithread chat_server_multithread.c
huophuc1@Phuc:~$ ./chat_server_multithread
Chat Server Multithread đang lắng nghe ở cổng 9000...
>> Client [phuc] đã đăng nhập.
>> Client [alice] đã đăng nhập.
>> Client [phuc] đã ngắt kết nối.
>> Client [alice] đã ngắt kết nối.
[]

```

Server

<pre> huophuc1@Phuc:~\$ nc 127.0.0.1 9000 Vui lòng đăng nhập theo cú pháp 'client_id: client_name' abc xyz >> Sai cú pháp! Mời hợp lệ: 'alice: Alicetpuyen' phuc: CackaPhuc >> Đăng nhập thành công! Bạn có thể chat. Xin chào alice: Tôi biết [] </pre>	<pre> huophuc1@Phuc:~\$ nc 127.0.0.1 9000 Vui lòng đăng nhập theo cú pháp 'client_id: client_name' alice: a >> Đăng nhập thành công! Bạn có thể chat. phuc: Xin chào Tôi biết [] </pre>
--	---

Client

Bài 4:

Lập trình ứng dụng **telnet_server** ([slide 174](#)) sử dụng kỹ thuật multithread.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <sys/socket.h>
```

```
#include <arpa/inet.h>
```

```
#include <pthread.h>
```

```
int check_login(char *user, char *pass) {  
    FILE *f = fopen("database.txt", "r");  
    if (!f) return 0;  
    char f_user[64], f_pass[64];  
    while (fscanf(f, "%s %s", f_user, f_pass) == 2) {  
        if (strcmp(user, f_user) == 0 && strcmp(pass, f_pass) ==  
0) {  
            fclose(f);  
            return 1;  
        }  
    }
```

```

    }
    fclose(f);
    return 0;
}

void* client_handler(void *arg) {
    int client = *(int*)arg;
    free(arg);

    char *msg = "Vui long nhap tai khoan (Cach nhau boi dau
cach):\n";

    send(client, msg, strlen(msg), 0);

    char buf[1024];
    int logged_in = 0;
    while (1) {
        int bytes = recv(client, buf, sizeof(buf) - 1, 0);
        if (bytes <= 0) {
            printf(">> Client %d da ngat ket noi.\n", client);
            break;
        }
        buf[bytes] = '\0';
        buf[strcspn(buf, "\r\n")] = 0;
    }

```

```

    if (logged_in == 0) {
        char user[64], pass[64];

        if (sscanf(buf, "%s %s", user, pass) == 2 &&
check_login(user, pass)) {
            logged_in = 1;

            char *ok = ">> Dang nhap thanh cong! Ban co the
nhap lenh:\n";

            send(client, ok, strlen(ok), 0);
        } else {
            char *err = ">> Tai khoan hoac mat khau sai. Vui long
thu lai:\n";

            send(client, err, strlen(err), 0);
        }
    }
}
else {
    char cmd[1100];
    char out_filename[64];
    sprintf(out_filename, "out_%d.txt", client);
    sprintf(cmd, "%s > %s", buf, out_filename);
    system(cmd);
    FILE *f = fopen(out_filename, "r");

```

```

    if (f) {
        char out_buf[1024];
        int has_out = 0;
        while (fgets(out_buf, sizeof(out_buf), f)) {
            send(client, out_buf, strlen(out_buf), 0);
            has_out = 1;
        }
        if (!has_out) {
            char *no_res = "(Lenh khong tra ve ket qua)\n";
            send(client, no_res, strlen(no_res), 0);
        }
        fclose(f);
        remove(out_filename);
    }
}

close(client);
return NULL;
}

```

```
int main() {  
    int listener = socket(AF_INET, SOCK_STREAM,  
IPPROTO_TCP);  
    struct sockaddr_in addr;  
    addr.sin_family = AF_INET;  
    addr.sin_addr.s_addr = htonl(INADDR_ANY);  
    addr.sin_port = htons(9000);  
    bind(listener, (struct sockaddr *)&addr, sizeof(addr));  
    listen(listener, 10);  
    printf("Telnet Server Multithread dang lang nghe o cong  
9000...\n");  
    while(1){  
        int client_fd = accept(listener, NULL, NULL);  
        if (client_fd == -1) continue;  
        printf(">> Co client moi ket noi (ID: %d)\n", client_fd);  
        int *client_ptr = malloc(sizeof(int));  
        *client_ptr = client_fd;  
        pthread_t tid;  
        pthread_create(&tid, NULL, client_handler, client_ptr);  
        pthread_detach(tid);  
    }  
}
```

```
}  
  
return 0;  
  
}
```

```
h@huphuc10:~$ gcc -pthread -o telnet_server_multithread telnet_server_multithread.c  
h@huphuc10:~$ ./telnet_server_multithread  
Telnet Server Multithread đang lang nghe o cong 9000...  
>> Co client moi ket noi (ID: 4)  
>> Co client moi ket noi (ID: 5)  
>> Client 4 da ngat ket noi.  
>> Client 5 da ngat ket noi.  
!
```

Server

```
h@huphuc10:~$ nc -l 27.0.0.1 9000  
Vui long nhap tai khoan (Cach nhau boi dau cach):  
abc xyz  
>> Tai khoan hoặc mật khẩu sai. Vui long thu lai:  
admin admin  
>> Đang nhập thành công! Ban có thể nhập lệnh:  
ls -l  
total 668  
-rw-r--r-- 1 huphuc10 huphuc10 16712 Apr 28 08:57 28_04_01  
-rw-r--r-- 1 huphuc10 huphuc10 3841 Apr 28 08:57 28_04_01.c  
-rw-r--r-- 1 huphuc10 huphuc10 16656 Apr 28 09:11 28_04_02  
-rw-r--r-- 1 huphuc10 huphuc10 2136 Apr 28 09:10 28_04_02.c  
-rw-r--r-- 1 huphuc10 huphuc10 20920 May 3 11:42 28_04_03  
-rw-r--r-- 1 huphuc10 huphuc10 5437 May 3 11:32 28_04_03.c  
-rw-r--r-- 1 huphuc10 huphuc10 16880 Apr 13 16:06 chat_server  
-rw-r--r-- 1 huphuc10 huphuc10 4536 Apr 13 16:02 chat_server.c  
-rw-r--r-- 1 huphuc10 huphuc10 17168 May 21 15:07 chat_server_multithread  
-rw-r--r-- 1 huphuc10 huphuc10 3486 May 21 15:07 chat_server_poll  
-rw-r--r-- 1 huphuc10 huphuc10 3812 Apr 20 17:24 chat_server_poll.c  
-rw-r--r-- 1 huphuc10 huphuc10 39 Apr 13 16:55 database.txt  
-rw-r--r-- 1 huphuc10 huphuc10 20944 Apr 6 08:07 email_server  
-rw-r--r-- 1 huphuc10 huphuc10 4604 Mar 31 09:13 email_server.c  
-rw-r--r-- 1 huphuc10 huphuc10 17888 May 21 14:01 file_server  
-rw-r--r-- 1 huphuc10 huphuc10 3188 May 21 14:01 file_server.c  
-rw-r--r-- 1 huphuc10 huphuc10 16616 May 11 15:17 http_server_preforking  
-rw-r--r-- 1 huphuc10 huphuc10 1380 May 10 10:46 http_server_preforking.c  
-rw-r--r-- 1 huphuc10 huphuc10 16672 May 21 14:40 http_server_prethread  
-rw-r--r-- 1 huphuc10 huphuc10 1340 May 21 14:39 http_server_prethread.c  
-rw-r--r-- 1 huphuc10 huphuc10 16064 Mar 24 21:43 info_client  
-rw-r--r-- 1 huphuc10 huphuc10 1767 Mar 24 21:50 info_client.c  
-rw-r--r-- 1 huphuc10 huphuc10 16480 Mar 24 21:43 info_server  
-rw-r--r-- 1 huphuc10 huphuc10 1607 Mar 24 21:50 info_server.c  
-rw-r--r-- 1 huphuc10 huphuc10 405 May 10 10:17 out.txt  
-rw-r--r-- 1 huphuc10 huphuc10 0 May 21 15:24 out_4.txt  
-rw-r--r-- 1 huphuc10 huphuc10 16704 May 21 14:23 pair_chat  
-rw-r--r-- 1 huphuc10 huphuc10 2132 May 21 14:10 pair_chat.c  
-rw-r--r-- 1 huphuc10 huphuc10 512 May 21 13:52 shared_folder  
-rw-r--r-- 1 huphuc10 huphuc10 16448 Mar 24 21:58 streaming_client  
-rw-r--r-- 1 huphuc10 huphuc10 1476 Mar 24 21:57 streaming_client.c  
-rw-r--r-- 1 huphuc10 huphuc10 16616 Mar 24 21:58 streaming_server  
-rw-r--r-- 1 huphuc10 huphuc10 2170 Mar 24 21:56 streaming_server.c  
-rw-r--r-- 1 huphuc10 huphuc10 16560 Apr 13 16:56 telnet_server  
-rw-r--r-- 1 huphuc10 huphuc10 4645 Apr 13 16:55 telnet_server.c  
-rw-r--r-- 1 huphuc10 huphuc10 17080 May 10 10:15 telnet_server_multiprocessing  
-rw-r--r-- 1 huphuc10 huphuc10 2950 May 10 10:14 telnet_server_multiprocessing.c  
-rw-r--r-- 1 huphuc10 huphuc10 17168 May 21 15:10 telnet_server_multithread  
-rw-r--r-- 1 huphuc10 huphuc10 3105 May 21 15:10 telnet_server_multithread.c  
-rw-r--r-- 1 huphuc10 huphuc10 16968 Apr 20 17:52 telnet_server_poll  
-rw-r--r-- 1 huphuc10 huphuc10 3942 Apr 20 17:52 telnet_server_poll.c  
-rw-r--r-- 1 huphuc10 huphuc10 16880 May 11 15:24 time_server  
-rw-r--r-- 1 huphuc10 huphuc10 2849 May 11 15:24 time_server.c  
-rw-r--r-- 1 huphuc10 huphuc10 16920 May 21 14:49 time_server_multithread  
-rw-r--r-- 1 huphuc10 huphuc10 2191 May 21 14:48 time_server_multithread.c  
-rw-r--r-- 1 huphuc10 huphuc10 16556 Apr 6 08:15 udp_chat  
-rw-r--r-- 1 huphuc10 huphuc10 2247 Apr 6 08:15 udp_chat.c  
-rw-r--r-- 1 huphuc10 huphuc10 16536 Mar 24 22:27 udp_client  
-rw-r--r-- 1 huphuc10 huphuc10 1677 Mar 24 22:27 udp_client.c  
-rw-r--r-- 1 huphuc10 huphuc10 16360 Mar 24 22:15 udp_server  
-rw-r--r-- 1 huphuc10 huphuc10 1761 Mar 24 22:15 udp_server.c
```

Client

Bài 5:

Lập trình ứng dụng **time_server** ([slide 204](#)) sử dụng kỹ thuật multithread.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <sys/socket.h>
```

```
#include <arpa/inet.h>
```

```
#include <pthread.h>
```

```
#include <time.h>
```

```
void* time_handler(void *arg) {
```

```
    int client = *(int*)arg;
```

```
    free(arg);
```

```
    char buf[256];
```

```
    send(client, "Nhap lenh (VD: GET_TIME dd/mm/yyyy):\n",  
37, 0);
```

```
    while(1){
```

```
        int bytes = recv(client, buf, sizeof(buf) - 1, 0);
```

```
        if (bytes <= 0) break;
```

```

buf[bytes] = '\0';
buf[strcspn(buf, "\r\n")] = 0;
char cmd[32] = "", format[32] = "";
if(sscanf(buf, "%s %s", cmd, format) == 2 &&
strcmp(cmd, "GET_TIME") == 0) {
    time_t t = time(NULL);
    struct tm *tm_info = localtime(&t);
    char time_str[64] = "";
    int valid = 1;

    if(strcmp(format, "dd/mm/yyyy") == 0)
strftime(time_str, sizeof(time_str), "%d/%m/%Y\n", tm_info);

    else if(strcmp(format, "dd/mm/yy") == 0)
strftime(time_str, sizeof(time_str), "%d/%m/%y\n", tm_info);

    else if(strcmp(format, "mm/dd/yyyy") == 0)
strftime(time_str, sizeof(time_str), "%m/%d/%Y\n", tm_info);

    else if(strcmp(format, "mm/dd/yy") == 0)
strftime(time_str, sizeof(time_str), "%m/%d/%y\n", tm_info);

    else valid = 0;

    if (valid) send(client, time_str, strlen(time_str), 0);
    else send(client, "Format khong ho tro.\n", 21, 0);
} else {

```

```

        send(client, "Sai cu phap.\n", 13, 0);
    }
}
close(client);
return NULL;
}

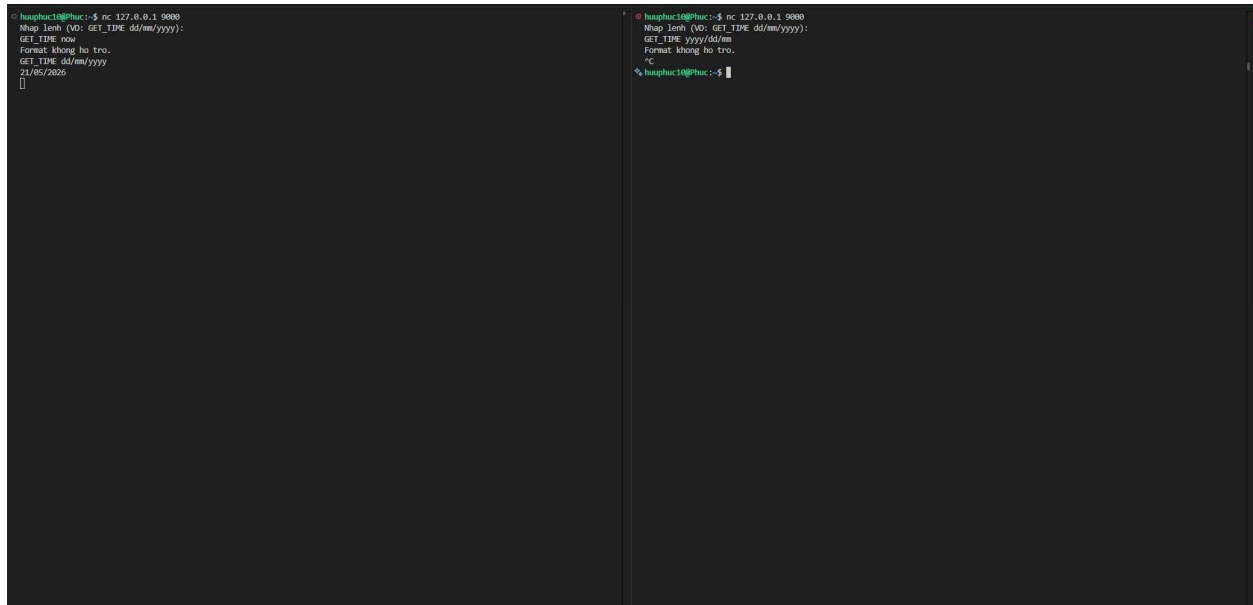
int main() {
    int listener = socket(AF_INET, SOCK_STREAM,
IPPROTO_TCP);
    struct sockaddr_in addr;
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons(9000);
    bind(listener, (struct sockaddr *)&addr, sizeof(addr));
    listen(listener, 10);
    printf("Time Server Multithread dang chay o cong 9000...\n");
    while(1){
        int client = accept(listener, NULL, NULL);
        if (client == -1) continue;
        int *client_ptr = malloc(sizeof(int));

```

```
    *client_ptr = client;
    pthread_t tid;
    pthread_create(&tid, NULL, time_handler, client_ptr);
    pthread_detach(tid);
}
return 0;
}
```

```
huuphuc10@huc:~$ gcc -pthread -o time_server_multithread time_server_multithread.c
huuphuc10@huc:~$ ./time_server_multithread
Time Server Multithread dang chay o cong 9080...
[]
```

Server



Client

Bài 6:

Lập trình ứng dụng **http_server** ([slide 154](#)) sử dụng kỹ thuật prethreading.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <sys/socket.h>
```

```
#include <arpa/inet.h>
```

```
#include <pthread.h>
```

```
void* worker_thread(void *arg) {
```

```

int listener = *(int*)arg;
char buf[2048];
while(1){
    int client = accept(listener, NULL, NULL);
    int ret = recv(client, buf, sizeof(buf) - 1, 0);
    if (ret > 0) {
        char *html = "HTTP/1.1 200 OK\r\nContent-Type:
text/html\r\n\r\n<html><body><h1>Xin chao! Toi la
Prethreading Server.</h1></body></html>";
        send(client, html, strlen(html), 0);
        printf(">> Luong (Thread ID: %ld) vua phuc vu 1 yeu
cau.\n", pthread_self());
    }
    close(client);
}
return NULL;
}

int main() {
    int listener = socket(AF_INET, SOCK_STREAM,
IPPROTO_TCP);
    struct sockaddr_in addr;

```

```
addr.sin_family = AF_INET;
addr.sin_addr.s_addr = htonl(INADDR_ANY);
addr.sin_port = htons(8080);
bind(listener, (struct sockaddr *)&addr, sizeof(addr));
listen(listener, 10);
printf("HTTP Prethreading chay tai http://127.0.0.1:8080\n");
int num_threads = 8;
pthread_t thread_id;
for (int i = 0; i < num_threads; i++) {
    pthread_create(&thread_id, NULL, worker_thread,
&listener);
    pthread_detach(thread_id);
}
while(1) { sleep(100); }
return 0;
}
```

```
huophuc1@Phuc:~$ gcc -pthread -o http_server_prethread http_server_prethread.c
huophuc1@Phuc:~$ ./http_server_prethread
HTTP Prethreading chạy tại http://127.0.0.1:8080/
>> Luong (Thread ID: 140687508117696) vua phuc vu 1 yeu cau.
>> Luong (Thread ID: 140687559296704) vua phuc vu 1 yeu cau.
>> Luong (Thread ID: 140687508117696) vua phuc vu 1 yeu cau.
>> Luong (Thread ID: 140687542388416) vua phuc vu 1 yeu cau.
>> Luong (Thread ID: 140687542388416) vua phuc vu 1 yeu cau.
>> Luong (Thread ID: 140687533934272) vua phuc vu 1 yeu cau.
>> Luong (Thread ID: 140687559296704) vua phuc vu 1 yeu cau.
>> Luong (Thread ID: 140687508117696) vua phuc vu 1 yeu cau.
>> Luong (Thread ID: 140687508571840) vua phuc vu 1 yeu cau.
>> Luong (Thread ID: 140687508571840) vua phuc vu 1 yeu cau.
>> Luong (Thread ID: 140687508571840) vua phuc vu 1 yeu cau.
>> Luong (Thread ID: 140687525488128) vua phuc vu 1 yeu cau.
>> Luong (Thread ID: 140687508117696) vua phuc vu 1 yeu cau.
>> Luong (Thread ID: 140687508117696) vua phuc vu 1 yeu cau.
>> Luong (Thread ID: 140687525488128) vua phuc vu 1 yeu cau.
>> Luong (Thread ID: 140687533934272) vua phuc vu 1 yeu cau.
>> Luong (Thread ID: 140687517025984) vua phuc vu 1 yeu cau.
>> Luong (Thread ID: 140687517025984) vua phuc vu 1 yeu cau.
>> Luong (Thread ID: 140687542388416) vua phuc vu 1 yeu cau.
>> Luong (Thread ID: 140687558842560) vua phuc vu 1 yeu cau.
>> Luong (Thread ID: 140687533934272) vua phuc vu 1 yeu cau.
>> Luong (Thread ID: 140687559296704) vua phuc vu 1 yeu cau.
>> Luong (Thread ID: 140687533934272) vua phuc vu 1 yeu cau.
[]
```

Server

← → ↻ http://127.0.0.1:8080/

Xin chao! Toi la Prethreading Server.

Test client bằng cách mở <http://127.0.0.1:8080>